“DESIGN AND VERIFICATION OF TRUE DUAL-PORT RAM USING VERILOG HDL”

Testing and Verification of Digital Circuits

3EC605ME24

By

Dhameliya Mitanshu (23BEC042)

Dhameliya Vasu (23BEC043)

Department of Electronics and Communication Engineering,

Institute of Technology,

Nirma University,

Ahmedabad 382 481

March 2026

Abstract— Dual Port RAM (Random Access Memory) is an essential component in modern digital systems where simultaneous data access is required. Unlike single port memory, dual port RAM allows two independent ports to perform read and write operations concurrently, thereby improving system performance and efficiency. In this project, a parameterized Dual Port RAM is designed using Verilog Hardware Description Language (HDL). The design supports configurable address width and memory depth, making it flexible for various applications. Two independent ports, Port A and Port B, are implemented with separate control signals, enabling simultaneous operations. A comprehensive testbench is developed to verify the functionality of the design. The testbench includes file-based stimulus generation, self-checking mechanisms, and multiple verification scenarios such as cross-port access, collision handling, simultaneous read/write operations, and reset behavior. Simulation results demonstrate correct operation for basic cases but reveal certain limitations in handling simultaneous access and collision scenarios. The project highlights the importance of proper synchronization and arbitration logic in memory design.

Keywords— True Dual-Port RAM, Verilog HDL, Synchronous Memory, Write Collision, FPGA, Testbench Verification.

1. Introduction

Memory is a fundamental component in every digital system. Conventional Single-Port RAM allows only one operation—either a read or a write—per clock cycle. When multiple modules need simultaneous memory access, this creates a bottleneck that reduces overall system throughput. True Dual-Port RAM (TDPRAM) overcomes this limitation by providing two fully independent ports, Port A and Port B, each capable of performing a read or write on every clock cycle without stalling the other.

TDPRAM is widely used in applications such as digital signal processing (DSP), multi-core processors, FPGA-based embedded systems, and communication data buffers where high-bandwidth concurrent memory access is essential. In FPGA devices, on-chip Block RAM primitives natively support the true dual-port configuration, making the design directly synthesizable.

This report documents the complete design, implementation, and functional verification of a parameterized synchronous TDPRAM module in Verilog HDL. The module is verified through a comprehensive self-checking testbench that covers all key operational scenarios—basic read/write, cross-port data integrity, write collision, simultaneous access at different addresses, and post-reset behavior.

2. THEORY

2.1 RAM Architecture Classification

RAM architectures are classified based on the number and capability of their access ports:

- **Single-Port RAM:** One port for both read and write. Only one operation can occur per clock cycle.
- **Simple Dual-Port RAM:** Port A is dedicated to write operations and Port B is dedicated to read operations. Partial parallelism is achieved, but neither port can switch roles.
- **True Dual-Port RAM:** Both ports are symmetric—each can independently perform read or write operations on every clock cycle. This design implements a True Dual-Port RAM.

2.2 True Dual-Port RAM Architecture

A TDPRAM consists of a single shared physical memory array accessed through two completely independent paths. Each port has its own address bus, data input bus, data output bus, and write-enable signal, while sharing a common clock and reset. Because both ports access the same memory array, simultaneous access to the same address requires a defined arbitration policy.

On any rising clock edge, the possible access combinations are: Port A writes while Port B reads; Port A reads while Port B writes; both ports read (no conflict); both ports write to different addresses (no conflict); or both ports write to the same address, which is a *collision* and must be handled deterministically.

2.3 Synchronous Operation

All operations in this design are synchronous—registered on the positive clock edge (`posedge clk`). Write operations latch data into memory on the rising edge when write-enable is asserted. Read operations capture the memory content into a registered output on the rising edge when write-enable is de-asserted, producing a one-cycle read latency. The synchronous active-high reset clears all memory locations to zero on the next rising edge, ensuring the system starts in a known state.

2.4 Write Collision Policy

A write collision occurs when both ports simultaneously assert their write-enable signals targeting the same address. This design resolves collisions deterministically by granting Port A priority—Port A's data is written and Port B's data is discarded. This ensures the memory state is always

predictable after a collision. Table 1 summarizes all port combinations and resulting behaviour.

Table 1. *Write Operation Truth Table*

we_a	we_b	Addresses Equal?	Behaviour
1	0	—	Write `data_a` to `mem[add_a]`
0	1	—	Write `data_b` to `mem[add_b]`
0	0	—	Read both ports independently
1	1	Yes (Collision)	Port A priority: write `data_a` to `add_a`
1	1	No	Write `data_a` to `add_a` AND `data_b` to `add_b`

3. Design Description

3.1 Module Parameters

The `Dual_Port` module is parameterized to allow easy scaling without modifying the core logic. The default configuration provides a 5-bit address space with 32 locations of 8-bit width (256 bits total). Table 2 lists all parameters.

Table 2. *Design Parameter Summary*

Parameter	Default Value	Description
`ADDR_WIDTH`	5	Address bus width — supports $2^5 = 32$ locations
`DEPTH`	32	Total number of memory words
Data Width	8-bit	Fixed width of each memory word
Clock	posedge clk	All operations on rising clock edge
Reset	Active-High	Synchronous reset — clears all memory to `0x00`
Write Priority	Port A	Port A data wins on simultaneous same-address writes

3.2 Port Interface

The module exposes separate address, data, and control signals for each port. Table 3 describes all interface signals.

Table 3. *Module Signal Interface*

Signal	Width	Direction	Description
`clk`	1-bit	Input	System clock
`rst`	1-bit	Input	Synchronous active-high reset
`we_a` / `we_b`	1-bit	Input	Write enable: 1 = write, 0 = read
`add_a` / `add_b`	5-bit	Input	Address bus for each port
`data_a` / `data_b`	8-bit	Input	Write data for each port
`read_a` / `read_b`	8-bit	Output (reg)	Registered read data output

4. Design Implementation

4.1 Memory Array and Reset

The shared memory is declared as a two-dimensional register array with `DEPTH` words, each 8 bits wide:

```
reg [7:0] mem [DEPTH-1:0];
```

On reset, a `for` loop iterates through all `DEPTH` locations and clears each to zero on the rising clock edge. This synchronous clear guarantees that after reset de-assertion the entire memory is in a known, deterministic state.

4.2 Dual Always Block Architecture

The design uses two separate `always` blocks—one for Port A and one for Port B—both triggered on `posedge clk`. Each block independently evaluates its port's write-enable signal. If write-enable is asserted, the data input is written to the addressed memory location. If write-enable is de-asserted, the memory content is captured into the registered output port, producing a one-cycle read latency.

4.3 Port A Operation

Port A's always block unconditionally writes `data_a` to `mem[add_a]` whenever `we_a` is high. No collision check is required in Port A's block because Port A always takes priority by design. On the read path, `mem[add_a]` is registered into `read_a`.

4.4 Port B Operation and Collision Handling

Port B's always block includes the write-collision detection logic. When `we_b` is asserted and the addresses match (`add_a == add_b`), the block writes `data_a`—not `data_b`—to `mem[add_a]`, ensuring Port A's data prevails. When the addresses differ, Port B writes `data_b` to `mem[add_b]` normally. On the read path, if addresses match, `read_a` is updated; otherwise `read_b` captures `mem[add_b]`.

5. Testbench Description

5.1 Overview

The testbench (`tb_Dual_Port`) instantiates the `Dual_Port` DUT with matching parameters (`ADDR_WIDTH = 5`, `DEPTH = 32`) and applies stimuli through a set of structured tasks. A 10 ns clock period (5 ns half-period) is used.

An `expected_mem[]` register array mirrors the anticipated memory state at all times. After every read operation, the DUT output is compared against `expected_mem[]`. Any mismatch increments an integer error counter and is printed to the console with the failing address, expected value, and observed value. At the end of simulation, the total error count and a PASS/FAIL verdict are displayed. A timeout watchdog at 200,000 ns ensures the simulation terminates even in the event of an unexpected hang.

5.2 File-Based Stimulus

At simulation start, the testbench generates two text files—`port_a_stim.txt` and `port_b_stim.txt`. Each line encodes three hexadecimal fields: write-enable, address, and data. Dedicated tasks (`apply_port_a_from_file`, `apply_port_b_from_file`) read these files and apply each vector to the DUT sequentially. This approach separates stimulus data from testbench logic, making it straightforward to modify test vectors without recompiling.

Port A stimulus covers 8 write operations at addresses `0x00`–`0x1F` followed by 8 read-back operations. Port B stimulus covers 8 writes at addresses `0x05`–`0x1E` followed by 8 read-backs.

5.3 Test Tasks

Table 4 summarizes all testbench tasks and their verification purpose.

Table 4. Testbench Task Summary

Task	Purpose
write_stimulus_files	Generates port_a_stim.txt and port_b_stim.txt with predefined hex vectors at simulation start.
apply_reset	Asserts rst for 2 clock cycles and clears expected_mem[] to all zeros.
apply_port_a_from_file	Applies 8 writes followed by 8 read operations on Port A and verifies outputs.
apply_port_b_from_file	Applies 8 writes followed by 8 read operations on Port B and verifies outputs.
cross_port_verify	Writes data via Port A and reads back through Port B to verify cross-port integrity.
collision_test	Both ports write to the same address simultaneously; verifies Port A priority.
sim_rw_diff_addr	Port A writes while Port B reads a different address, verifying parallel access.
post_reset_check	After reset, verifies all sampled memory locations return 0x00.

5.4 Test Sequence

The main `initial` block executes the following ordered sequence: (1) generate stimulus files; (2) apply first reset; (3) Port A file stimulus—8 writes then 8 read-backs; (4) Port B file stimulus—8 writes then 8 read-backs; (5) cross-port verify at addresses `0x0B` and `0x0C`; (6) collision test at address `0x0D` with `0x1A` (Port A) and `0x2B` (Port B); (7) simultaneous write on Port A at `0x0E` while reading `0x0A` on Port B; (8) second reset and post-reset checks on addresses `0x00`, `0x0F`, `0x10`, and `0x1F` on both ports.

6. Simulation Results

6.1 Basic Read/Write Verification

Port A and Port B file-based stimulus phases executed without errors. All 8 write operations for each port completed correctly, and all 8 subsequent read-back operations returned values matching the expected memory model, confirming the basic synchronous write and registered read paths are functionally correct for both ports.

6.2 Cross-Port Data Integrity

Writing `0xDE` to address `0x0B` via Port A and reading `0x0B` via Port B returned the correct value `0xDE`. The same result was confirmed for address `0x0C` with value `0xAD`. This verifies that data written by one port is immediately accessible through the other port in the following clock cycle.

6.3 Write Collision

The collision test simultaneously asserted `we_a` and `we_b` at address `0x0D` with values `0x1A` and `0x2B` respectively. Reading address `0x0D` after the collision returned `0x1A`, confirming Port A priority is correctly enforced.

6.4 Post-Reset Verification

After the second reset, all sampled addresses on both ports returned `0x00`, confirming the synchronous reset correctly clears the entire memory array regardless of previously written data.

6.5 Results Summary

Table 5 summarizes the pass/fail result for every test scenario. The final console output reported **Errors = 0** and **PASS**.

Table 5. Simulation Results Summary

Test Scenario	Result	Notes
Port A file-based write and read	**PASS**	All 8 write/read pairs correct
Port B file-based write and read	**PASS**	All 8 write/read pairs correct
Cross-port verify (0x0B, 0x0C)	**PASS**	Port B correctly reads Port A data
Write collision at 0x0D	**PASS**	Port A data `0x1A` retained
Post-reset check	**PASS**	All locations return `0x00`
Overall — Errors = 0	**PASS**	Full simulation passes

7. Advantages and Limitations

7.1 Advantages

- **Parameterized design:** Easily scalable to any memory size by changing `ADDR_WIDTH` and `DEPTH`.
- **Full dual-port access:** Both ports independently read or write every clock cycle, doubling effective memory bandwidth.
- **Deterministic collision resolution:** Port A priority ensures predictable memory state after simultaneous writes.
- **Fully synchronous:** Glitch-free operation and direct FPGA BRAM compatibility.
- **Self-checking testbench:** Automated verification with file-based stimulus and expected memory model.

7.2 Limitations

- Two-always-block structure may introduce simulation race conditions; merging into a single block is more robust.
- Single clock domain only—does not support clock-domain-crossing scenarios requiring independent clocks per port.
- No byte-enable support—only full 8-bit word writes are possible.
- Simultaneous read and write to the same address: read output during a write is implementation-dependent.

8. Conclusion

This project successfully demonstrated the design and functional verification of a synchronous True Dual-Port RAM in Verilog HDL. The parameterized module supports two fully independent read/write ports on a shared memory array with a deterministic Port-A-priority collision policy and synchronous reset. The comprehensive self-checking testbench validated all key functional scenarios—basic read/write, cross-port data integrity, write collision handling, simultaneous access at different addresses, and post-reset verification—with zero errors reported. The design is synthesizable and directly applicable to FPGA Block RAM primitives, making it suitable for real embedded and digital system applications. Future work includes adding independent port clocks for clock-domain-crossing support, byte-enable signals for partial word writes, and formal verification using SystemVerilog assertions.

REFERENCES

[1] S. Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, 2nd ed. Prentice Hall, 2003.

[2] Intel Corporation, *Dual-Port RAM with Synchronous Read*, FPGA Application Note, Intel Altera Documentation.

[3] Xilinx Inc., *UG901 — Vivado Design Suite User Guide: RTL Synthesis, Memory Coding Guidelines*.

[4] IEEE, *IEEE Std 1364-2001 — IEEE Standard for the Verilog Hardware Description Language*, 2001.

[5] Digital Design Laboratory Course Material, Department of EC Engineering, Nirma University, Ahmedabad, 2024–25.